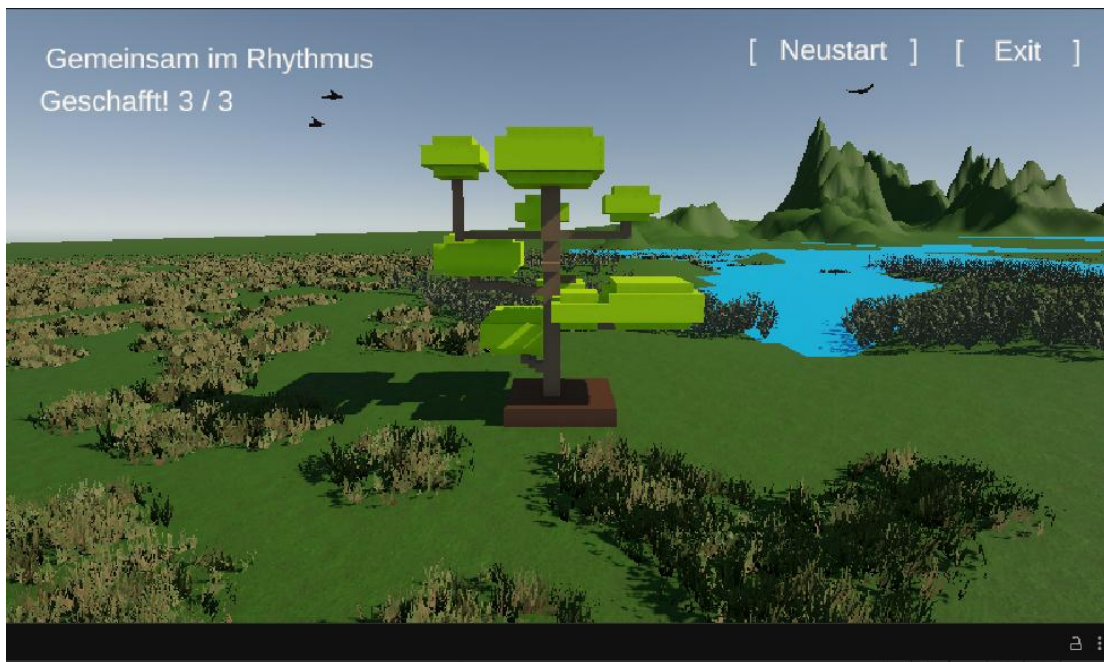


TWO HEARTS

Projektdokumentation XR & Biofeedback



Susan Jaturat

Entwurf · User Stories · Implementierung · Entwicklerdokumentation

Inhaltsübersicht

1. Projektkontext und Ausgangssituation

2. Projektidee und Zielsetzung

3. Anforderungen

4. Entwurf und Interaktionskonzept

5. User Stories und Akzeptanzkriterien

6. Technisches Konzept

7. Entwicklerdokumentation

8. Datenmodell und Atemsimulation

9. Programmablauf

10. Testkonzept und Testdaten

11. Projektverlauf

12. Grenzen und Weiterentwicklung

13. Fazit

Anhang: Dateistruktur und Dokumentationsnachweise

1. Projektkontext und Ausgangssituation

Das Projekt entstand im Rahmen einer zweiwöchigen SPE-Phase im dualen Studium bei Siemens. Der thematische Rahmen lautet XR und Biofeedback. Innerhalb der Projektphase sollte zunächst ein Projektantrag erstellt, anschließend ein Entwurf mit Storyboard bzw. User Stories ausgearbeitet und schließlich ein funktionsfähiger Prototyp umgesetzt und dokumentiert werden. Als Entwicklungsumgebung wurde Unity eingesetzt.

Der Prototyp „Two Hearts“ verbindet eine geführte Atemübung mit visuellem Biofeedback. Zwei Personen sollen ihren Atemrhythmus an eine pulsierende Lichtkugel angleichen. Im realen Zielsystem würden dafür Messdaten zweier Nutzer eingelesen. Im vorliegenden Prototyp werden diese Messdaten bewusst durch eine vorbereitete JSON-Datei simuliert.

2. Projektidee und Zielsetzung

Ziel von „Two Hearts“ ist eine ruhige, kooperative Meditationserfahrung. Nicht die Leistung einer einzelnen Person, sondern die gemeinsame **Synchronität steht im Mittelpunkt**. Die Lichtkugel gibt den Soll-Atemrhythmus vor. Beide simulierten Werte der Personen werden gleichzeitig mit diesem Sollwert verglichen. Nur wenn beide ausreichend synchron atmen, gilt ein Atemzyklus als erfolgreich.

Als langfristiges visuelles Feedback wächst ein Baum. Für jede erfolgreiche Runde wird die nächste Wachstumsstufe angezeigt. Sobald drei erfolgreiche Atemzyklen erreicht sind, wird das Finale ausgelöst. Im aktuellen Stand wird ein Endtext angezeigt; weitere geplante Elemente sind ein rosa Overlay über den Himmel, Schmetterlinge und zusätzliche Naturdetails wie Blumen und Meditationsmusik.

3. Anforderungen

3.2 Funktionale Anforderungen

ID	Anforderung	Umsetzung im Prototyp
F1	Meditation über Start-Button beginnen	StarteMeditation() im MeditationsManager
F2	Countdown vor Beginn anzeigen	5-Sekunden-Countdown per Coroutine
F3	Sonnenaufgang weich einblenden	Interpolation der Lichtintensität
F4	Atemkugel nach Intro anzeigen	Aktivierung nach Sonnenaufgang
F5	Atemrhythmus 5 s ein / 8 s aus abbilden	13-Sekunden-Zyklus in Simulation
F6	Daten zweier Personen einlesen	p1_value und p2_value aus JSON
F7	Beide Personen mit Sollwert vergleichen	Toleranzprüfung pro Messpunkt
F8	10 Atemzyklen auswerten	anzahlAtemzyklen = 10
F9	3 gemeinsame Erfolge als Ziel	benoetigteErfolge = 3
F10	Fortschritt textuell anzeigen	Atemzug- und Erfolgsanzeige via TextMeshPro
F11	Baum bei Erfolg wachsen lassen	ErfolgreicheRunde()
F12	Finale nach Erreichen des Ziels	FinaleAblauf()
F13	Meditation zurücksetzen	Neustart()
F14	Anwendung über Exit beenden	SpielBeenden() / Application.Quit()

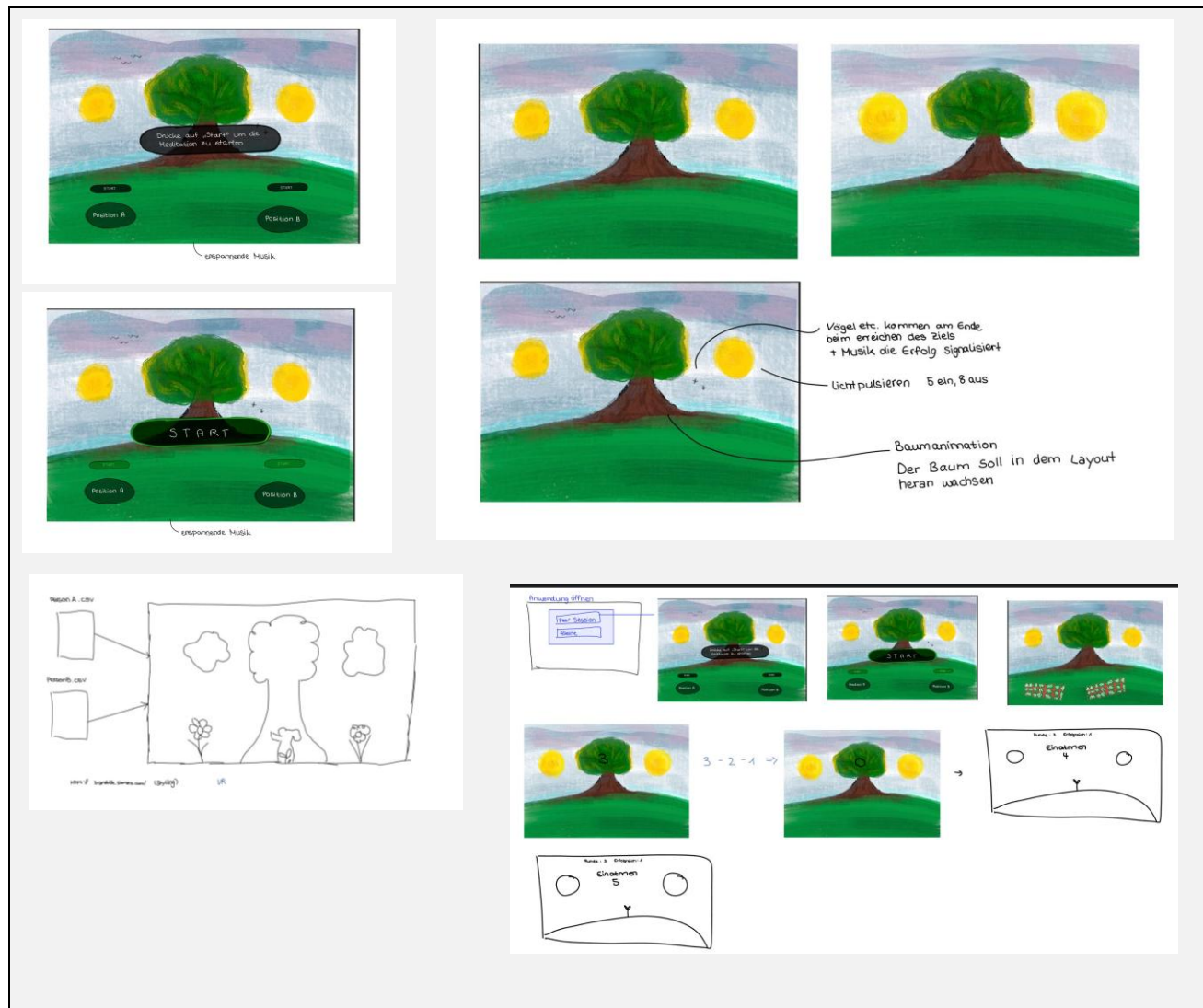
3.3 Nichtfunktionale Anforderungen

Die Anwendung soll ruhig, verständlich und visuell zurückhaltend wirken. Übergänge sollen weich erfolgen, insbesondere das Pulsieren der Atemkugel und das Einblenden des Lichts. Für den Prototyp steht Nachvollziehbarkeit vor technischer Komplexität: Messdaten werden deshalb reproduzierbar aus einer Datei „breathing_simulation“ abgespielt.

```
users > jatur > Two Hearts > Assets > Data > breathing_simulation.js  
{"data": [  
  {  
    "time": 0.0,  
    "target_value": 0,  
    "p1_value": 0.025,  
    "p2_value": 0.3  
  },  
  {  
    "time": 0.5,  
    "target_value": 0.028,  
    "p1_value": 0.003,  
    "p2_value": 0.328  
  },  
  {  
    "time": 1.0,  
    "target_value": 0.104,  
    "p1_value": 0.129,  
    "p2_value": 0.404  
  }  
],  
}
```

4. Entwurf und Interaktionskonzept

Die Interaktion ist bewusst linear gehalten. Zu Beginn sieht der Nutzer die ruhige Szene und einen **Start-Button**. Nach dem Start folgt ein **Countdown**. Anschließend wird die Szene durch den Sonnenaufgang heller. Erst danach erscheinen Atemkugel und Fortschrittsinformationen. Die **Kugel** dient als kontinuierliches **Atemsignal**: größere bzw. stärker leuchtende Kugel entspricht einem höheren Atemwert, kleinere Kugel einem niedrigeren Atemwert.



5. User Stories und Akzeptanzkriterien

User Story	Akzeptanzkriterien
Als Nutzer möchte ich die Meditation über einen klaren Start-Button beginnen, damit ich selbst den Startzeitpunkt bestimme.	Beim Klick startet ein 5-Sekunden-Countdown; der Start-Button wird anschließend ausgeblendet.
Als Nutzer möchte ich einen ruhigen Übergang in die Meditation erleben.	Nach dem Countdown wird die Sonne über mehrere Sekunden eingeblendet; danach erscheinen Atemkugel und Simulation.
Als Nutzer möchte ich erkennen können, wann ich ein- und ausatmen soll.	Die Lichtkugel verändert ihre Größe und Leuchtstärke flüssig entsprechend dem Soll-Atemwert.
Als Nutzer möchte ich sehen, ob wir gemeinsam synchron waren.	Ein Atemzyklus zählt nur dann als erfolgreich, wenn die erforderliche Trefferquote für beide Personen erreicht wird.
Als Nutzer möchte ich meinen Fortschritt erkennen.	Atemzugnummer und gemeinsame Erfolge werden im UI angezeigt.
Als Nutzer möchte ich für gemeinsame Synchronität belohnt werden.	Bei erfolgreicher Runde wächst der Baum um eine Stufe; nach drei Erfolgen startet das Finale.
Als Nutzer möchte ich die Anwendung verlassen können.	Der Exit-Button beendet im Build die Anwendung; im Unity-Editor wird der Play-Modus beendet.

6. Technisches Konzept

Die Implementierung ist in **mehrere C#-Komponenten** aufgeteilt. Der **MeditationsManager** übernimmt die Ablaufsteuerung. Der **BreathingSimulationManager** ist für das Laden und zeitgerechte Abspielen der JSON-Daten sowie für die Bewertung der Synchronität verantwortlich. **AtemTaktSteuerung** visualisiert den Sollwert als pulsierende und leuchtende Kugel. **BaumWachstumFinale** setzt erfolgreiche Runden in sichtbare Wachstumsstufen um.

Komponente	Verantwortung	Wichtige Schnittstellen
MeditationsManager	Gesamtablauf, UI, Sonne, Erfolge, Finale, Neustart	StarteMeditation(), AtemzyklusBeendet(), SimulationBeendet(), Neustart()
BreathingSimulation-Manager	JSON laden, Messpunkte abspielen, Zyklen auswerten	StarteSimulation(), StoppeSimulation(), WerteZyklusAus()
BreathingData	Datenklassen für JSON-Deserialisierung	BreathingPoint, BreathingContainer
AtemTaktSteuerung	Skalierung und Leuchten der Atemkugel	SetzeAtemWert(), Zuruecksetzen()
BaumWachstumFinale	Erde erzeugen und Baumstufen darstellen	ErfolgreicheRunde(), Zuruecksetzen()
CameraFokus	Kamera relativ zum Baum positionieren	LateUpdate()
ExitButton	Anwendung/Play-Modus beenden	SpielBeenden()

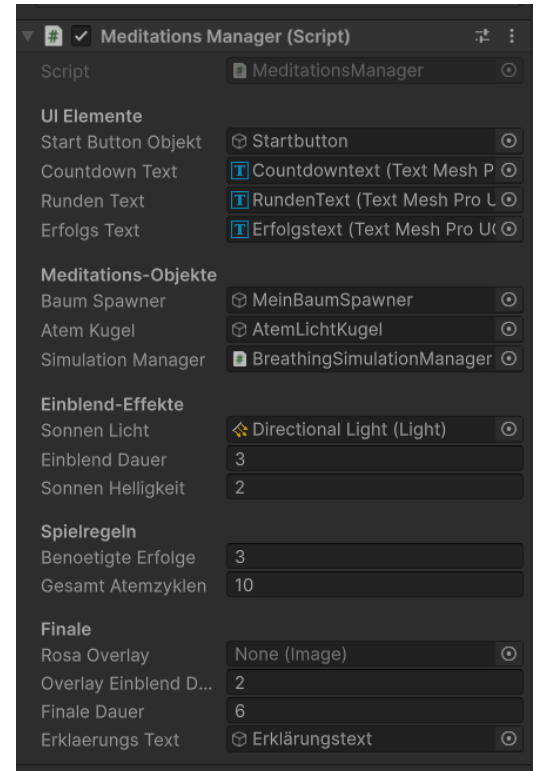


7. Entwicklerdokumentation

7.1 MeditationsManager

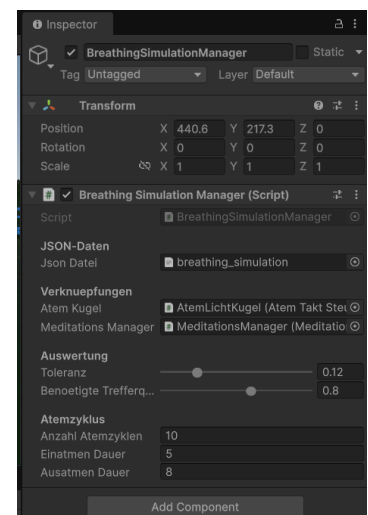
Der **MeditationsManager** ist die zentrale Ablaufsteuerung. Beim Start werden Baum und Atemkugel deaktiviert, die Sonnenintensität auf 0 gesetzt und UI-Texte geleert. *StarteMeditation()* setzt den Erfolgszähler zurück und startet *CountdownAblauf()*. Nach fünf Sekunden wird der Baum-Spawner aktiviert und *SanftesEinblendenRoutine()* führt den Sonnenaufgang durch. Anschließend wird die Atemkugel aktiviert und die Simulation gestartet.

Nach jedem ausgewerteten Atemzyklus ruft der SimulationManager **AtemzyklusBeendet()** auf. Bei Erfolg wird der Zähler erhöht und *BaumWachstumFinale.ErfolgreicheRunde()* aufgerufen. Sobald die konfigurierte Anzahl von drei Erfolgen erreicht ist, startet *FinaleAblauf()*: Simulation stoppen, Atemkugel ausblenden, Finaltext anzeigen und nach der konfigurierten Finaldauer *Neustart()* ausführen.



7.2 BreathingSimulationManager

Der **BreathingSimulationManager** liest eine **TextAsset-JSON-Datei** über JsonUtility ein. Das Skript akzeptiert sowohl einen Container mit dem Feld „data“ als auch ein direktes JSON-Array, das intern automatisch in einen Container verpackt wird. Vor dem Start werden JSON-Datei, Atemkugel und MeditationsManager auf gültige **Referenzen** geprüft.



Die Messpunkte werden anhand ihres Zeitstempels in Echtzeit abgespielt. Ein Zyklus dauert 13 Sekunden (5 Sekunden Einatmen + 8 Sekunden Ausatmen). Für jeden Messpunkt wird geprüft, ob sowohl **p1_value** als auch **p2_value** innerhalb der Toleranz vom **target_value** liegen. Die Trefferquote eines Zyklus wird mit benoetigteTrefferquote verglichen. Standardmäßig beträgt die Toleranz 0,12 und die erforderliche Trefferquote 80 %.

StarteSimulation(): Startet die Atemsimulation und lädt die benötigten Atemdaten. Sie bildet den Einstiegspunkt der Datenverarbeitung.

```
public void StartSimulation()
{
    Debug.Log("BreathingSimulationManager: StartSimulation() wurde aufgerufen.");
    if (simulationLaufte)
    {
        Debug.LogWarning("BreathingSimulationManager: Simulation läuft bereits.");
        return;
    }
    if (simulationCoroutine != null)
    {
        StopCoroutine(simulationCoroutine);
        simulationCoroutine = null;
    }
    if (!LadeDaten())
    {
        Debug.LogError("BreathingSimulationManager: Simulation konnte nicht gestartet werden.");
        return;
    }
    if (atemKugel == null)
    {
        Debug.LogError("BreathingSimulationManager: Keine Atemkugel zugewiesen.");
        return;
    }
    if (meditationsManager == null)
    {
        Debug.LogError("BreathingSimulationManager: Kein MeditationsManager zugewiesen.");
        return;
    }
    Debug.Log("BreathingSimulationManager: JSON erfolgreich geladen. Messpunkte: " +
        "data.data.Count");
    simulationCoroutine = StartCoroutine(SimulationAblauf());
}
```


LadeDaten(): Liest die simulierten Atemdaten aus der JSON-Datei ein und wandelt sie in BreathingPoint-Objekte um. Für eine spätere Datenbankbindung kann hier die JSON-Datei durch eine andere Datenquelle ersetzt werden.

SimulationsAblauf(): Die wichtigste Methode für die eigentliche Atemlogik. Sie verarbeitet die Messpunkte nacheinander, übergibt den target_value an die Atemkugel und prüft, ob die Werte von Person 1 und Person 2 innerhalb der vorgegebenen Toleranz zum Sollwert liegen.

WertezyklusAus(): Wertet einen vollständigen Atemzyklus aus. Aus dem Verhältnis erfolgreicher Messpunkte zu allen Messpunkten wird die Trefferquote berechnet. Anschließend wird über *meditationsManager.AtemzyklusBeendet(...)* das Ergebnis an den MeditationsManager weitergegeben.

```
1 Verweis
private IEnumerator SimulationsAblauf()
{
    simulationLaeuft = true;

    Debug.Log("BreathingSimulationManager: SimulationsAblauf gestartet.");

    atemKugel.externeSteuerung = true;
    atemKugel.Zuruecksetzen();

    float zyklusDauer = einatmenDauer + ausatmenDauer;

    int aktuellerZyklus = 0;
    int trefferImZyklus = 0;
    int messpunkteImZyklus = 0;

    float simulationsStart = Time.time;

    for (int i = 0; i < daten.data.Count; i++)
    {
        BreathingPoint punkt = daten.data[i];

        float sollZeit = simulationsStart + punkt.time;

        while (Time.time < sollZeit)
        {
            yield return null;
        }

        int punktZyklus =
            Mathf.FloorToInt(punkt.time / zyklusDauer);

        punktZyklus =
            Mathf.Clamp(
                punktZyklus,
```

7.3 AtemTaktSteuerung

Die Atemkugel erhält über SetzeAtemWert() einen normierten Wert zwischen 0 und 1. SmoothDamp und SmoothStep glätten die Übergänge. Aus dem geglätteten Wert werden die Skalierung zwischen minimaleGroesse und maximaleGroesse, die Intensität eines untergeordneten Point Lights sowie die Material-Emission berechnet. Dadurch pulsiert die Kugel flüssiger und wirkt selbstleuchtend.

```
3 Verweise
public void SetzeAtemWert(float wert)
{
    zielAtemWert = Mathf.Clamp01(wert);
}
```

```
1 Verweis
public void Zuruecksetzen()
{
    aktuelleStufe = 0;

    if (aktuellerBaum != null)
    {
        Destroy(aktuellerBaum);
        aktuellerBaum = null;
    }

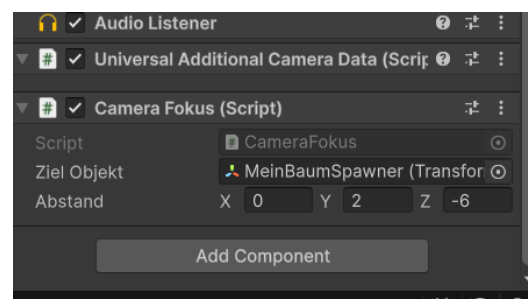
    ErstelleErde();
}
```

7.4 BaumWachstumFinale

Das Baumskript verwaltet eine Liste von Baumstufen-Prefabs. Zu Beginn wird ein einfaches braunes Erdobjekt erzeugt. Bei jeder erfolgreichen Runde wird die bisherige Baumstufe entfernt und die nächste Stufe instanziiert. Die Größe wird abhängig vom Fortschritt zwischen startGroesse und endGroesse interpoliert. Zuruecksetzen() entfernt den aktuellen Baum und setzt die Stufe wieder auf 0.

7.5 Kamera und Exit

CameraFokus hält die Kamera relativ zum Baum-Spawner und richtet sie in LateUpdate() auf einen Punkt oberhalb des Zielobjekts aus. ExitButton trennt Editor- und Build-Verhalten: Im Editor wird der Play-Modus beendet, im fertigen Build wird Application.Quit() aufgerufen.



8. Datenmodell und Atemsimulation

Jeder Messpunkt enthält **vier Werte**: time (Zeitstempel in Sekunden), target_value (Sollwert der Atemkugel), **p1_value** (simulierter Messwert Person 1) und **p2_value** (simulierter Messwert Person 2). Alle Atemwerte liegen im Bereich 0 bis 1. Die JSON-Datei enthält Messpunkte im Abstand von 0,5 Sekunden.

Feld *in BreathingData*	Bedeutung	Beispiel
time	Zeitpunkt seit Simulationsstart	15.5s
target_value	Soll-Atemwert / Vorgabe der Kugel	0.5s
p1_value	simulierter Atemwert Person 1	0.475s
p2_value	simulierter Atemwert Person 2	0.535s

```
1 using System;
2 using System.Collections.Generic;
3
4 [System.Serializable]
5 public class BreathingPoint
6 {
7     public float time;
8     public float target_value;
9     public float p1_value;
10    public float p2_value;
11 }
12
13 [System.Serializable]
14 public class BreathingContainer
15 {
16     public List<BreathingPoint> data;
17 }
18
```

8.1 Erfolgslogik

Ein einzelner Messpunkt zählt als Treffer, wenn beide Personen gleichzeitig höchstens 0,12 vom Sollwert abweichen. Ein vollständiger Atemzyklus gilt als erfolgreich, wenn mindestens 80 % seiner Messpunkte Treffer sind. Der Prototyp benötigt insgesamt drei erfolgreiche Zyklen.

Die bereitgestellte Testdatei ist gezielt so aufgebaut, dass die Erfolge nicht unmittelbar am Anfang auftreten. Bei den aktuellen Parametern sind Atemzyklus 2, 4 und 6 erfolgreich. Damit wird der dritte gemeinsame Erfolg erst im sechsten Atemzyklus erreicht und das Finale ausgelöst.

Atemzyklus	Messpunkte	Trefferquote	Ergebnis
1	26	0 %	nicht erfolgreich
2	26	100 %	erfolgreich
3	26	0 %	nicht erfolgreich
4	26	100 %	erfolgreich
5	26	0 %	nicht erfolgreich
6	26	100 %	erfolgreich → 3/3 erreicht

```
{
  "data": [
    {
      "time": 0.0,
      "target_value": 0,
      "p1_value": 0.025,
      "p2_value": 0.1
    },
    {
      "time": 0.5,
      "target_value": 0.028,
      "p1_value": 0.003,
      "p2_value": 0.328
    },
    {
      "time": 1.0,
      "target_value": 0.104,
      "p1_value": 0.129,
      "p2_value": 0.404
    }
  ]
}
```

9. Programmablauf

1. Anwendung befindet sich im Startzustand.
2. Nutzer klickt auf Start.
3. Fünfsekündiger Countdown wird angezeigt.
4. Start-Button verschwindet und der Baum-Spawner wird aktiviert.
5. Sonnenlicht wird über mehrere Sekunden eingeblendet.
6. Atemkugel und Fortschrittstexte erscheinen.



7. JSON-Simulation startet und steuert die Kugel über target_value.
8. p1_value und p2_value werden fortlaufend mit target_value verglichen.
9. Nach jedem 13-Sekunden-Zyklus wird die Trefferquote berechnet.
10. Bei Erfolg wächst der Baum und der Erfolgszähler steigt.
11. Beim dritten Erfolg startet das Finale.
12. Nach der Finaldauer wird die Anwendung in den Startzustand zurückgesetzt.

```
using System;
using System.Collections.Generic;

[System.Serializable]
2 Versionen
public class BreathingPoint
{
    3 Versionen
    public float time;
    4 Versionen
    public float target_value;
    5 Versionen
    public float p1_value;
    6 Versionen
    public float p2_value;
}

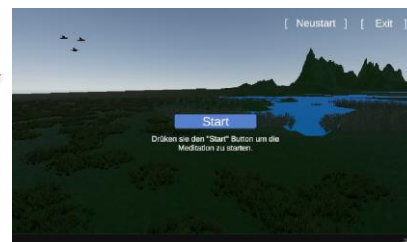
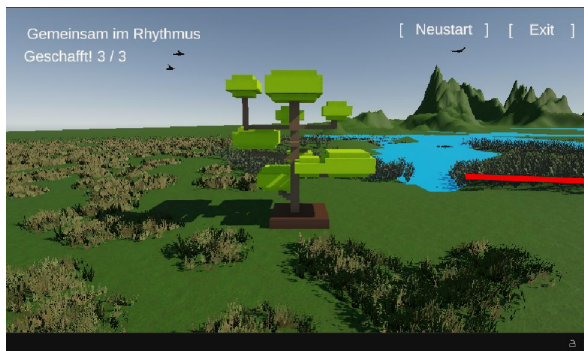
[System.Serializable]
2 Versionen
public class BreathingContainer
{
    6 Versionen
    public List<BreathingPoint> data;
}
```

```
3 Versionen
private void WerteZyklusAusf(
    int zyklusIndex,
    int treffer,
    int gesamt
)
{
    float quote =
        gesamt > 0
        ? (float)treffer / gesamt
        : 0f;

    bool erfolgreich =
        quote >= benötigteTrefferquote;

    Debug.Log(quote);

    meditationManager.AtemzyklusBeendet(
        zyklusIndex + 1,
        erfolgreich
    );
}
```



10. Testkonzept und Testdaten

Testfall	Erwartetes Verhalten
Start-Button	Countdown startet; Button ist nicht mehrfach auslösbar.
Countdown	5, 4, 3, 2, 1 wird zeitlich nacheinander angezeigt.
Sonnenaufgang	Lichtintensität steigt weich von 0 auf den konfigurierten Zielwert.
JSON-Laden	Datei wird gelesen; bei fehlender Datei erscheint Debug-Fehler.
Atemkugel	Skalierung und Emission folgen dem target_value weich.
Nicht synchrone Runde	Erfolgszähler bleibt unverändert; Baum wächst nicht.
Synchrone Runde	Erfolgszähler steigt; Baum wechselt zur nächsten Stufe.
3. Erfolg	Finale startet; Simulation wird gestoppt.
Neustart	Baum, Kugel, Texte, Sonne, Overlay und Erfolgsstand werden zurückgesetzt.
Exit	Editor: Play-Modus stoppt; Build: Anwendung wird geschlossen.

Für die Testbarkeit wurden Debug-Ausgaben eingebaut. Der SimulationManager protokolliert unter anderem das Laden der JSON-Datei, einzelne Messwerte, Trefferquoten pro Atemzyklus sowie Start und Ende der Simulation.

11. Projektverlauf

Die Projektphase war in zwei Wochen gegliedert. In der ersten Woche lagen die Schwerpunkte auf XR-Einführung, Biofeedback-Recherche, Projektantrag, XR-Design, Storyboard und Prototyping. In der zweiten Woche folgten XR-Programmierung, Einrichtung der Entwicklungsumgebung, Unity-Grundlagen, 3D-Modelle, Finalisierung, Integration und Dokumentation. Den Abschluss bildet die XR-Vernissage mit Video, Live-Demonstration und Dokumentation.

Tag	Schwerpunkt
1	XR-Intro, Recherche Biofeedback, Projektantrag
2	XR-Intro, Genehmigung Projektantrag, erste Schritte
3	XR-Design, Storyboard
4	XR-Design, Prototypen
5	Zwischenbilanz
6	XR-Programmierung I, IDE-Einrichtung
7	XR-Programmierung II, Unity/A-Frame
8	XR-Programmierung III, 3D-Modelle
9	XR-Programmierung IV, Finalisierung und Dokumentation
10	XR-Vernissage: Video, Live-Demo, Dokumentation

12. Grenzen und Weiterentwicklung

Der aktuelle Stand ist ein funktionaler Prototyp und verwendet keine echten Biosensoren. Die Atemdaten beider Personen werden aus einer vorbereiteten JSON-Datei eingelesen. Dadurch lässt sich die Interaktion reproduzierbar demonstrieren, ohne dass Hardwareintegration erforderlich ist.

Mögliche nächste Entwicklungsschritte sind:

Anbindung realer Atem- oder Biosensoren statt statischer JSON-Testdaten.

Getrennte Echtzeit-Datenströme für zwei Nutzer.

Kalibrierung der Atemwerte pro Person.

Schmetterlingsanimationen im Finale.

Rosa Himmelsfärbung als echtes 3D-/Skybox-Element hinter Baum und Landschaft statt als UI-Overlay.

Blumen und weitere Naturdetails als zusätzliche visuelle Belohnung.

Einbindung meditativer Hintergrundmusik und Lautstärkeregelung.

Robustere Zustandsmaschine für Start, Intro, Meditation, Finale und Reset.

13. Fazit

„Two Hearts“ demonstriert das Grundprinzip eines kooperativen Biofeedback-Systems in Unity. Der Prototyp verbindet eine geführte Atemvorgabe mit simulierten Messdaten zweier Personen und übersetzt Synchronität in unmittelbar sichtbares Feedback. Die Aufteilung in Datenmodell, Simulationslogik, Visualisierung und zentrale Ablaufsteuerung ermöglicht es, die simulierten Daten später schrittweise durch reale Sensordaten zu ersetzen.

Anhang A – Dateistruktur

Datei	Zweck
AtemTaktSteuerung.cs	Visualisierung des Atem-Sollwerts durch Größe, Licht und Emission.
BaumWachstumFinale.cs	Verwaltung der Baumstufen und Reset.
BreathingData.cs	Serialisierbare Klassen für JSON-Daten.
BreathingSimulationManager.cs	Einlesen, zeitliches Abspielen und Auswerten der Atemdaten.
CameraFokus.cs	Kameraausrichtung auf den Baum-Spawner.
ExitButton.cs	Beenden der Anwendung bzw. des Play-Modus.
MeditationsManager.cs	Zentrale Ablauf- und UI-Steuerung.
breathing_simulation.json	Simulierte Atemmessdaten zweier Personen und Sollwerte.

Anhang B – Noch einzufügende Projektnachweise

Für die finale Abgabe sollten an den markierten Stellen eigene Screenshots bzw. Entwurfsartefakte ergänzt werden. Besonders sinnvoll sind: Projektantrag, QoC/Entscheidungsdocumentation, Wireframes, Storyboard, Unity-Hierarchy, Inspector-Verknüpfungen, laufende Meditation, Baumstufen und Finalzustand.

QoC - Question, Options & Criteria

Design & Software - Architekturdur : Questions, Options, Criteria

QoC

- How to Manipulate ; How to Travel
- Warum VR warum AR

Questions (Fragen / Probleme)

1. How to manipulate?

Wie lange Warte vom Sensor-Input des Controllers bis zur visuellen Umsetzung in der Engine? (Zielzeit Home QoC < 20 ms)

Physikalische Konsistenz, wie reagiert System, wenn Nutzer versucht, virtuelle Hand in Wand zu stecken?

2. How to travel?

Transportierung / Raumkonzepte

Sicherheitsräume (Chaperone / Guardian)

Kamerakontrolle

Orientierungshilfe

Wie berät Nutzer nach Teleportieren die Orientierung

3. Warum VR?

Ich möchte die Realität komplett ersetzen um mögliche Störfaktoren auszublenden und so eine kontrollierte, beruhigende Umgebung zu schaffen.

4. Wenn Schweben Menüs, sind Sie in

einem komfortablen Les-Entfernung 1,5-3 Meter. Folgen Blick des Nutzers verzögert statt klar am Kopf

5. Pixelgröße groß genug (Pixelation)

6. Haptisches Feedback

Options (Optionen)

- Hand geht durch
 - Hand prüft ob
- Raum für Sitzen, stehen oder umhergehen ausgeklübelt für sitzen
- Kopf niemals schnell bewegen - Motion sickness
- Beibehalten der Blickrichtung, oder Spüren auf dem Boden

Vibration

Criteria (Kriterien)

Benutzerfreundlichkeit

Konsistenz

Design

Komfort

Komfort

Komfort

Komfort

Benutzerfreundlichkeit

Question, Options & Criteria - Two Hearts

1. Question

Wie kann ein kooperatives Spielerlebnis gestaltet werden, bei dem zwei Personen durch das gemeinsame Angleichen ihrer Atmung einen Baum vom Samen bis zum vollständig gewachsenen Baum wachsen lassen?

Das Ziel des Projekts „Two Hearts“ ist es, die Zusammenarbeit und Synchronisation zwischen zwei Personen spielerisch erfahrbar zu machen. Beide Spieler verfügen über eine eigene Lichtkugel, die ihre Atmung symbolisiert. Durch eine möglichst gleichmäßige und synchronisierte Atmung sollen die beiden Lichtkugeln miteinander harmonisieren.

Die gemeinsame Synchronisation wirkt sich direkt auf das Wachstum eines Baumes aus. Je besser die beiden Spieler ihre Atmung aufeinander abstimmen, desto stärker wächst der Baum. Das Wachstum dient dabei als sichtbare Rückmeldung für die erreichte Synchronisation.

Die zentrale Frage für die technische und gestalterische Umsetzung lautet daher:

Wie soll die Atem-Synchronisation der beiden Spieler erkannt und dargestellt werden und wie soll sie das Wachstum des Baumes beeinflussen?

2. Options

Für die Umsetzung der Atem-Synchronisation und des Baumwachstums wurden verschiedene Varianten betrachtet.

Option 1: Synchronisation über einen festen Rhythmus

Bei dieser Variante wird für beide Spieler ein bestimmter Atemrhythmus vorgegeben. Die Lichtkugeln bewegen sich beispielsweise gleichzeitig auf und ab oder werden entsprechend des vorgegebenen Rhythmus größer und kleiner.

Die Spieler versuchen, ihre Atmung an diesen vorgegebenen Rhythmus anzupassen. Je näher beide Spieler am gleichen Rhythmus liegen, desto stärker wächst der Baum.

Vorteile:

- relativ einfach umzusetzen
- für die Spieler leicht verständlich
- gut kontrollierbar
- keine aufwendige Erfassung der tatsächlichen Atmung notwendig
-

Nachteile:

- die tatsächliche Atmung der Spieler wird nicht erfasst
- das Spielgefühl kann eher wie das Befolgen eines Taktes wirken
- die persönliche Synchronisation zwischen den beiden Spielern wird nur indirekt dargestellt

Option 2: Synchronisation über die tatsächliche Atmung

Bei dieser Variante wird die tatsächliche Atmung der beiden Spieler erfasst. Jede Person erhält eine eigene Lichtkugel, deren Bewegung die jeweilige Atmung visualisiert.

Die beiden Lichtkugeln bewegen sich entsprechend des individuellen Atemrhythmus. Atmen beide Personen unterschiedlich schnell oder zu unterschiedlichen Zeitpunkten, bewegen sich die Kugeln asynchron.

Je stärker sich die Atemrhythmen angleichen, desto synchroner bewegen sich die Lichtkugeln. Erreichen beide Spieler eine ausreichend hohe Synchronisation, wächst der Baum.

Vorteile:

- starke Verbindung zwischen realer Handlung und Spiel
- die Kooperation der beiden Spieler wird direkt sichtbar
- die Lichtkugeln können als intuitive Visualisierung der Atmung dienen
- das Baumwachstum bietet eine direkte Belohnung für gute Zusammenarbeit
- das Konzept von „Two Hearts“ wird besonders deutlich

Nachteile:

- technische Umsetzung der Atemerkennung ist anspruchsvoller
- die Erfassung kann durch unregelmäßige Atmung oder technische Ungenauigkeiten beeinflusst werden
- eventuell zusätzliche Hardware oder Sensorik notwendig

Option 3: Manuelle Steuerung der Lichtkugeln

Bei dieser Variante wird die Atmung nicht direkt gemessen. Stattdessen steuert jeder Spieler seine eigene Lichtkugel beispielsweise über einen Controller, eine Taste oder eine andere Eingabemöglichkeit.

Die Spieler versuchen, ihre Lichtkugeln möglichst gleichzeitig zu bewegen. Wenn die Bewegungen übereinstimmen, steigt der Synchronisationswert und der Baum beginnt zu wachsen.

Vorteile:

- technisch einfach umzusetzen
- keine zusätzliche Atemsensoren notwendig
- zuverlässige Eingabe
- gut für einen ersten Prototyp geeignet

Nachteile:

- die eigentliche Atmung wird nicht berücksichtigt
- die Verbindung zwischen realer Atmung und Spielmechanik geht verloren
- die Spieler synchronisieren eher ihre Eingaben als ihre Atmung
- die ursprüngliche Idee des Projekts wird abgeschwächt

Option 4: Synchronisation über visuelle oder akustische Hinweise

Bei dieser Variante erhalten die Spieler zusätzliche Hinweise, um ihre Atmung aneinander anzupassen. Beispielsweise könnten die Lichtkugeln bei einer Annäherung der Atemrhythmen heller werden oder sich stärker miteinander verbinden.

Zusätzlich könnten Töne oder andere audiovisuelle Signale eingesetzt werden. Bei guter Synchronisation entsteht beispielsweise ein harmonischer Klang oder ein stärkeres Leuchten der beiden Kugeln.

Das Baumwachstum bleibt dabei die übergeordnete Belohnung.

Vorteile:

- gute Verständlichkeit für die Spieler
- unmittelbares Feedback
- kann eine ruhige und immersive Atmosphäre schaffen
- unterstützt die Spieler dabei, die richtige Synchronisation zu finden

Nachteile:

- zusätzliche Gestaltung und Programmierung notwendig
- zu viele Hinweise könnten vom eigentlichen Erlebnis ablenken
- bei zu starken visuellen oder akustischen Effekten könnte die Atmosphäre des Spiels beeinträchtigt werden

3. Criteria

Um die verschiedenen Optionen miteinander vergleichen zu können, werden folgende Kriterien festgelegt:

1. Spielerische Verständlichkeit

Die Spieler sollen schnell verstehen, was sie tun müssen und welchen Einfluss ihre Atmung auf das Spiel hat.

2. Bezug zur ursprünglichen Spielidee

Die Umsetzung soll möglichst eng mit dem Grundgedanken von „Two Hearts“ verbunden sein. Zwei Personen müssen sich aufeinander einstellen und gemeinsam etwas entstehen lassen.

3. Technische Umsetzbarkeit

Die gewählte Lösung muss innerhalb des vorhandenen technischen Rahmens und der verfügbaren Entwicklungszeit realisierbar sein.

4. Immersion

Die Mechanik soll sich möglichst natürlich in die Spielwelt integrieren und nicht wie eine rein technische Aufgabe wirken.

5. Feedback

Die Spieler sollten unmittelbar erkennen können, wie gut sie synchronisiert sind. Die beiden Lichtkugeln und das Wachstum des Baumes sollen hierfür als visuelles Feedback dienen.

6. Kooperation

Die Mechanik soll die Zusammenarbeit zwischen den beiden Spielern fördern. Es soll nicht darum gehen, dass ein Spieler alleine erfolgreich ist, sondern dass beide gemeinsam das Ziel erreichen.

7. Erweiterbarkeit

Die gewählte technische Lösung sollte später erweitert werden können, beispielsweise durch verschiedene Wachstumsstufen des Baumes, unterschiedliche Synchronisationsgrade oder zusätzliche audiovisuelle Effekte.

4. Bewertung der Optionen

Kriterium	Fester Rhythmus	Tatsächliche Atmung	Manuelle Steuerung	Visuelle/akustische Hinweise
Spielerische Verständlichkeit	+++	++	+++	+++
Bezug zur Spielidee	++	+++	+	+++
Technische Umsetzbarkeit	+++	+	+++	++
Immersion	++	+++	+	+++
Feedback	++	+++	++	+++
Kooperation	++	+++	++	+++
Erweiterbarkeit	++	+++	++	+++

5. Entscheidung

Als bevorzugte Variante wird die **Synchronisation über die tatsächliche Atmung in Kombination mit visuellen und gegebenenfalls akustischen Rückmeldungen** gewählt.

Die beiden Spieler erhalten jeweils eine eigene Lichtkugel. Diese stellt die individuelle Atmung der jeweiligen Person dar. Durch die Bewegung, Helligkeit oder Größe der Lichtkugeln wird sichtbar, wie sich die beiden Atemrhythmen zueinander verhalten.

Atmen die beiden Spieler zunehmend synchron, reagieren die Lichtkugeln entsprechend und nähern sich beispielsweise in ihrer Bewegung oder ihrem Leuchten an. Gleichzeitig steigt der Synchronisationsgrad.

Bei ausreichender Synchronisation beginnt der Baum zu wachsen. Das Wachstum erfolgt dabei schrittweise, sodass die Spieler unmittelbar sehen können, dass ihre gemeinsame Anstrengung einen Fortschritt bewirkt.

Dadurch entsteht eine direkte Verbindung zwischen **Atmung + Synchronisation + Kooperation = Baumwachstum**.

Die Lichtkugeln übernehmen dabei eine zentrale Rolle, da sie sowohl die individuelle Atmung darstellen als auch den Synchronisationszustand für beide Spieler sichtbar machen.

Das finale Ziel ist, dass beide Spieler durch eine möglichst gute Synchronisation gemeinsam den Baum **vom Samen bis zum vollständig ausgewachsenen Baum** wachsen lassen.

Design 4: Software / Feedback / Sound / Hardware / Criteria			
Design	Software	Feedback	Sound
1. Spielmechanik	Die Spieler können ihre Atmung über einen Controller steuern. Die Lichtkugeln bewegen sich entsprechend der Atmung. Der Baum wächst, wenn die Spieler synchronisiert sind.	Die Spieler erhalten visuelle Rückmeldung über ihre Atmung. Die Lichtkugeln leuchten heller, wenn sie synchronisiert sind.	Die Spieler erhalten akustische Rückmeldung über ihre Atmung. Es gibt einen Ton, wenn sie synchronisiert sind.
2. Spielwelt	Die Spieler befinden sich in einer virtuellen Welt. Der Baum wächst in der Mitte der Welt. Die Spieler können den Baum berühren und ihn wachsen lassen.	Die Spieler können den Baum sehen und ihn wachsen lassen. Die Lichtkugeln leuchten heller, wenn sie synchronisiert sind.	Die Spieler können den Baum hören und ihn wachsen lassen. Es gibt einen Ton, wenn sie synchronisiert sind.
3. Spielmechanik	Die Spieler können ihre Atmung über einen Controller steuern. Die Lichtkugeln bewegen sich entsprechend der Atmung. Der Baum wächst, wenn die Spieler synchronisiert sind.	Die Spieler erhalten visuelle Rückmeldung über ihre Atmung. Die Lichtkugeln leuchten heller, wenn sie synchronisiert sind.	Die Spieler erhalten akustische Rückmeldung über ihre Atmung. Es gibt einen Ton, wenn sie synchronisiert sind.
4. Spielwelt	Die Spieler befinden sich in einer virtuellen Welt. Der Baum wächst in der Mitte der Welt. Die Spieler können den Baum berühren und ihn wachsen lassen.	Die Spieler können den Baum sehen und ihn wachsen lassen. Die Lichtkugeln leuchten heller, wenn sie synchronisiert sind.	Die Spieler können den Baum hören und ihn wachsen lassen. Es gibt einen Ton, wenn sie synchronisiert sind.